pyInnerBubble - examples overview

Royi Shabtay

2026-07-13

Abstract

pyInnerBubble is a python library written for inner flow measurements of bubbles (or droplets). The library is based on the method reported in XXX. This document presents the examples and is used as a tutorial. for this library.

Contents

1 Introduction **1**

2 Bubble shape and optic flow **2**

3 grid and data assimilation **4**

4 Scalar transport **6**

A Appendix **7**

1 Introduction

pyInnerBubble is based on the methodology reported in XXX. The method is divided into several stages and relies on synchronized PLIF/shadowgraphy measurements of a rising bubble. In the appendix, figure 9 illustrates the workflow of the library, with the colors indicating the different sections corresponding to the original article.

The bubble interface in both the PLIF and shadowgraph images is detected using the Segment Anything Model (SAM) algorithm, Kirillov et al. [2023]. Following the edge detection, the three-dimensional bubble shape is reconstructed. This reconstruction enables the mapping between the camera plane and the physical coordinate system. The internal velocity field is then obtained using the Farnebäck optical flow algorithm, Farnebäck [2003], and transformed into the physical coordinate system to compensate for optical distortions.

The library further enables augmentation of the measured velocity field at the bubble interface and smoothing of the reconstructed flow field using a data assimilation approach. In this framework, the axisymmetric Navier–Stokes equations are solved, and the tangential velocity distribution at the bubble interface is optimized to achieve the best agreement with the experimental measurements. Once the complete smoothed velocity field is obtained, the scalar

transport equation can be solved to evaluate the Sherwood/Nusselt number while neglecting the external mass/heat transfer resistance.

The first example in § 2 uses two pairs of PLIF/shadowgraph images to reconstruct the bubble shape and extract the internal flow field. In § 3, a computational mesh is generated using conformal mapping and employed for the data assimilation procedure based on the axisymmetric Navier–Stokes equations and the measured velocity field. Finally, in § 4, the scalar transport equation is solved numerically, and the Sherwood number is calculated for a range of Peclet numbers.

2 Bubble shape and optic flow

The first example demonstrates how to load a pair of images, reconstruct the bubble geometry, and extract the internal flow field. Figure 1 shows a pair of experimental images for a bubble with $\mathrm{Re}_g = 301$. The images are loaded using the following commands:

```
shadowgraph_image = str(SCRIPT_DIR / "shadowgraph0024.jpg")
plif_image = str(SCRIPT_DIR / "piv0024.jpg")
```

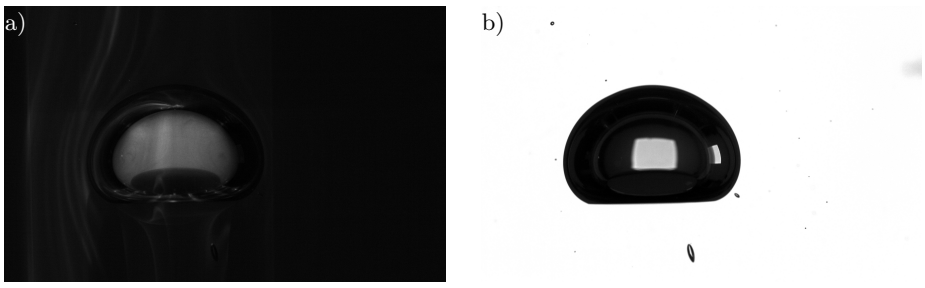


Figure 1: Images from experiment on $\mathrm{Re}_g = 301$ bubble. (a) PLIF (b) shadowgraph

The next step is to detect the bubble boundaries in both the shadowgraph and PLIF images. This is performed using the Meta Segment Anything Model (SAM). The library provides a wrapper around SAM, allowing the user to easily load the model and perform image segmentation.

First, specify the path to the SAM checkpoint:

```
SAM_CHECKPOINT = REPO_ROOT / "checkpoints" / "sam_vit_h_4b8939.pth
↪ "
```

Next, define the corresponding model type:

```
SAM_MODEL_TYPE = "vit_h" # must match the checkpoint (vit_h, vit_l
↪ , or vit_b)
```

SAM is a machine-learning-based segmentation model that requires the user to provide a small number of prompt points to identify the object of interest. Specifically, at least one foreground point (inside the bubble) and one background point (outside the bubble) must be supplied. SAM then generates several candidate segmentations and selects the one with the highest confidence score.

Figure 2 shows the resulting segmentation together with the selected foreground (green) and background (red) prompt points. In this example, the prompt

points are defined as

`shadowgraph_foreground = np.array([[614, 640]])`

and

`shadowgraph_background = np.array([[320, 192]])`.

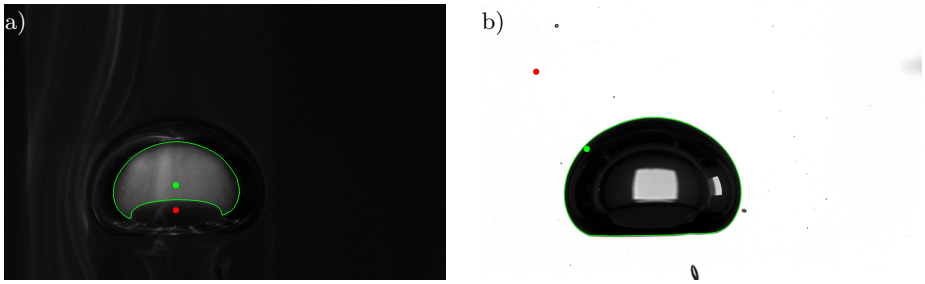


Figure 2: Segmented images with choosen background (red) and foreground (green) points. (a) PLIF (b) shadowgraph

After detecting the bubble boundaries, the bubble geometry is reconstructed, and both images are cropped and transformed into a reference frame that moves with the bubble. The crop size can be adjusted using the crop parameters passed to the `reconstruct_bubble_shape` function.

Next, a mapping between the camera image and the physical coordinate system is generated. In the provided example, the bubble reconstruction and camera mapping are performed in parallel using

`MAX_WORKERS = 4`.

The internal flow field is then computed using the Farnebäck optical flow algorithm. The algorithm parameters can be modified through

`FARNEBACK_PARAMS = FarnebackParams( num_pyramid_levels=3, ... )`.

After the optical flow calculation, the velocity field is smoothed using a two-dimensional Savitzky–Golay filter with a second-order polynomial. The smoothing radius is specified by

`SMOOTH_RADIUS = 7`.

Note that this parameter defines the radius of the filter rather than its window size. Consequently, the filter window is given by

window size $= 2 \times$ radius $+ 1$.

Because both the Farnebäck algorithm and the Savitzky–Golay filter use rectangular neighborhoods, velocity estimates near the bubble boundary are influenced by pixels outside the bubble. These edge regions should therefore be discarded by specifying an appropriate edge-removal width, for example,

`EDGE_REMOVAL = 15`.

Finally, the velocity field is mapped to the physical coordinate system, saved to disk (e.g., using `np.savetxt(flow_dir / "u_x.txt", ...)` and `np.savetxt(` ↪ `flow_dir / "u_y.txt", ...)`), and plotted, as shown in Figure 3.

The velocity field obtained from a single image pair is not symmetric, as the results presented in the accompanying publication. This is expected because the experimental measurements contain noise. To obtain a smoother and more symmetric flow field, average the velocity fields from multiple image pairs.

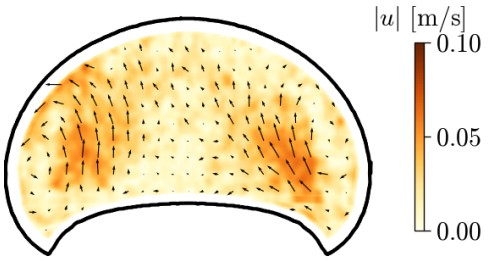


Figure 3: The velocity magnitude and a quiver overlay calculated from the example of two pairs of images

3 grid and data assimilation

The data assimilation procedure solves an inverse problem. Experimental velocity measurements are used to estimate the boundary conditions that reproduce the observed flow field. In the present implementation, the axisymmetric, dimensionless Navier–Stokes equations are solved in streamfunction form. The optimal boundary conditions are determined by minimizing the discrepancy between the numerical and experimental velocity fields using the Powell optimization algorithm Powell [1964].

Before performing data assimilation, a computational mesh must be generated. The library uses a conformal mapping from the physical bubble geometry to the unit disk, allowing the governing equations to be solved on a regular computational domain. An example of the mesh-generation procedure is provided in `grid_build_example.py`; however, any structured single-block mesh compatible with the solver may be used.

The bubble geometry must first be loaded and normalized. In the example, this is done using

```
BUBBLE_SHAPE_FILE = SCRIPT_DIR / "bubble_shape_tot.txt"
```

and

```
D_REF = 853.0.
```

The bubble shape is represented using a Fourier-series expansion. This representation provides a smooth, continuous description of the interface and avoids discontinuities that may degrade the quality of the conformal mapping and numerical solution.

The mesh-generation parameters are summarized in Table 1. To accurately capture the strong velocity gradients near the bubble rim (or skirt) region, the tangential coordinate distribution is represented using a piecewise function, allowing a higher concentration of grid points in regions of interest.

In the example, you can see the created mesh on the unit disk and the mapped computational mesh, figure 4.

After the mesh creation, the data assimilation can be run. In the examples folder, there are examples with and without the weighting matrix. Also, the examples are running on a very coarse grid. In real scientific/engineering problems, the mesh should be much finer and we suggest running the code on an HPC, for example, a 48-core machine, as we did in our cases.

The script `example_DA.py` demonstrates the complete data assimilation work-

Table 1: Mesh creation parameters

Name	Definition
N_THETA_HALF	half the number of angular nodes
N_RADIAL	number of radial nodes
BETA_THETA	larger = denser toward the angular ends
BETA_RADIAL	larger = denser toward the interface r=1
ALPHA	The split in piecewise function at $\theta = \alpha\pi$

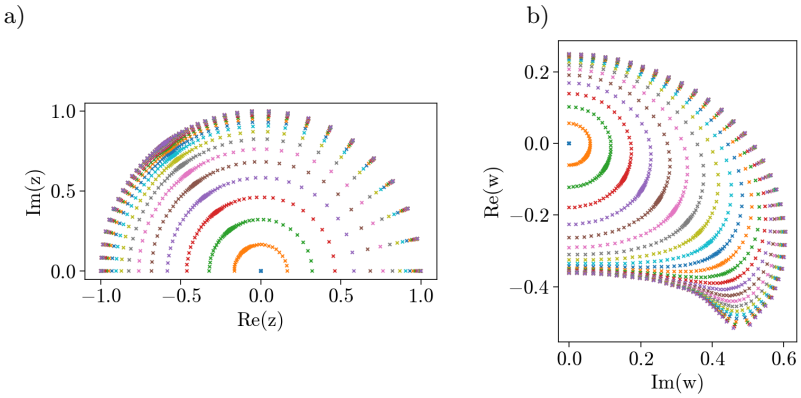


Figure 4: The mesh nodes in the unit disk (a), and mapped into the bubble shape (b).

flow, including the use of a weighting matrix. In this example, the weighting matrix is computed from the variance of the measured velocity over multiple image pairs, assigning lower weights to regions with higher experimental uncertainty.

Before running the assimilation, the experimental data must be normalized using the physical calibration parameters defined in

`# --- physical calibration (MKS) --`

The gas Reynolds number should also be specified, for example,

`REYNOLDS = 301.0`.

The mesh is being loaded in

`MESH_FILE = SCRIPT_DIR / "mesh_low.npz"`

and the experimental data in

`FLOW_FILE = SCRIPT_DIR / "mean_flow_exp8.npz"`

Since mesh generation and data assimilation are independent steps, the computational mesh must be aligned with the experimental data. In the example, the alignment is performed using

`z_max = np.max(z_mat)`

and

`cf_y = z_max - (y - y_top) / BUBBLE_DIAMETER_PX`.

The default output of the data assimilation process is shown in Figure 5. It includes:

- (a) the optimization history showing the reduction of the objective function

- (b) the reconstructed stream-function contours
- (c) the recovered tangential velocity distribution along the bubble surface

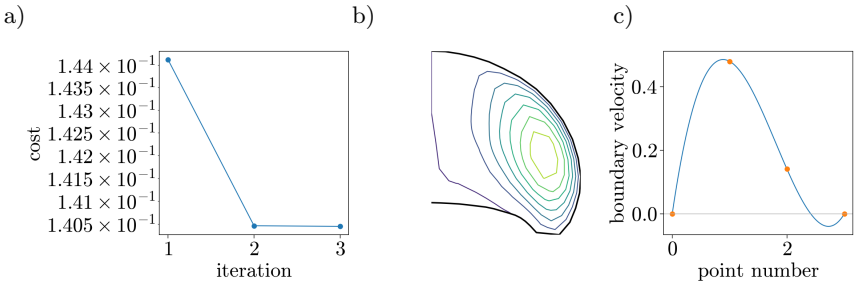


Figure 5: The default figure of the data assimilation process. a) Convergence of the optimization process, b) stream function iso-curves, and c) the tangential velocity at the bubble boundary

The algorithm also plots a comparison between the measured and assimilated velocity magnitude. For easier visual comparison, the measured flow field is mirrored so that both fields have the same orientation.

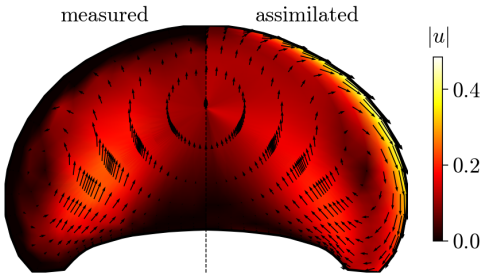


Figure 6: A comparison of the measured (left) and the assimilated (right) velocity magnitude

4 Scalar transport

The heat/mass transfer problem is solved using the assimilated flow field. The user defines the final simulation time, which determines when the diffusion simulation is terminated:

`T_END_DIFFUSION = 0.08`

The simulation uses the normalized diffusion time scale, $\hat{t} = \frac{t d_{eq}^2}{D}$

where D is the scalar diffusion coefficient and d_{eq} is the equivalent bubble diameter. The scalar transport problem is solved for a range of Peclet numbers defined by

`PE_VEC = [10, 60, 100, 300, 600, 1000]`.

The implemented solver assumes negligible external mass-transfer resistance. Therefore, the initial condition and boundary condition are defined as $\tilde{c}(\hat{t} = 0) = 1$, and $\tilde{c}(\hat{n} = 1) = 0$

where $\hat{n}$ is the normalized coordinate in the direction normal to the bubble interface, $\hat{n} = 1$ is the bubble edge.

Figure 7(a) shows the Sherwood number as a function of normalized time. As expected, the Sherwood number approaches an asymptotic value at long times, in agreement with the theoretical diffusion time scale ($\hat{t}_\infty \approx 1/(4\pi^2)$).

Figure 7(b) presents the asymptotic Sherwood number as a function of Peclet number. The two limiting regimes are clearly observed: the diffusion-dominated regime at $\mathrm{Pe} \to 0$ and the advection-dominated regime at $\mathrm{Pe} \to \infty$.

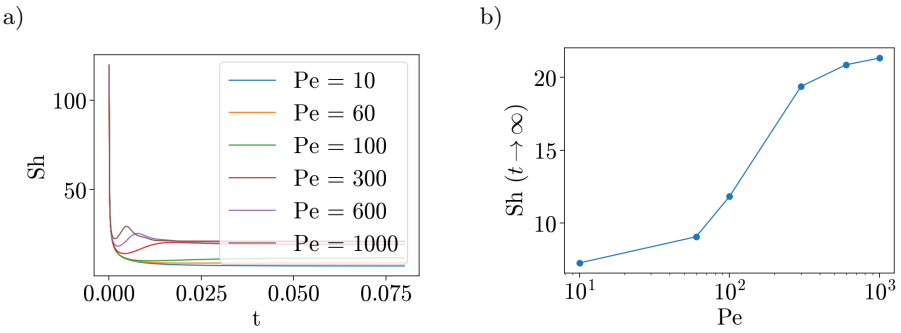


Figure 7: a) the Sherwood number vs diffusive time for various Pe numbers. b) The asymptotic Sherwood vs Pe number

The concentration normalized by the maximum concentration in the field at the asymptotic regime is given in figure 8.

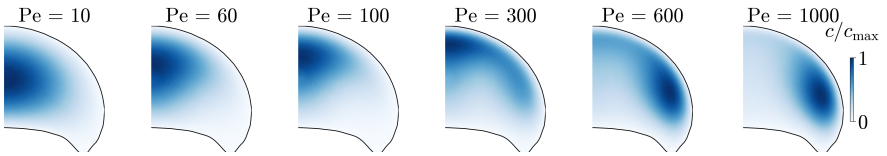


Figure 8: Concentration field for various Pe numbers at the asymptotic regime

A Appendix

Libarary flow chart

Figure 9 demonstrates the flow chart of the library with the required inputs and outputs. The color code is by the original article sections.

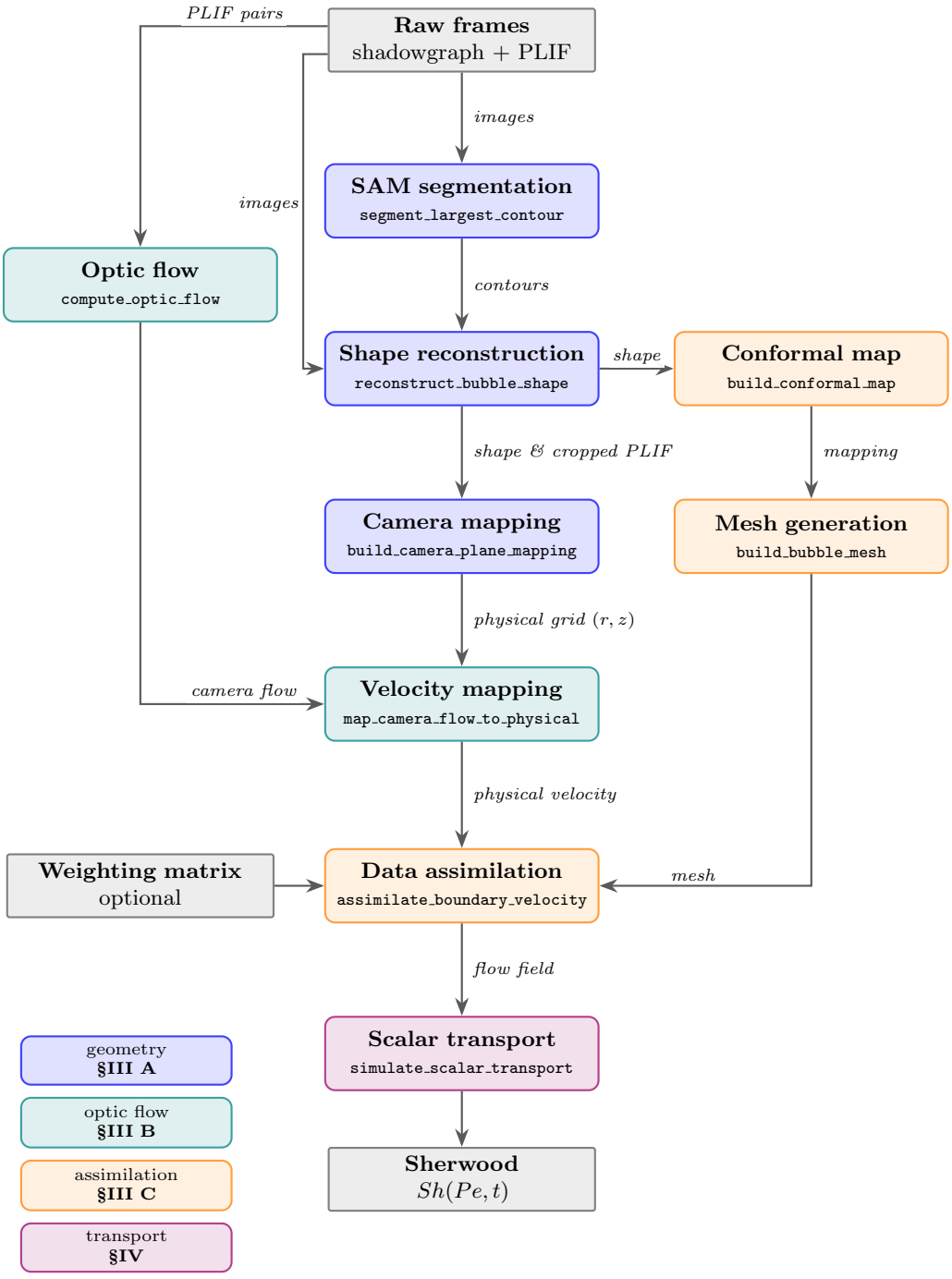


Figure 9: Flow chart of the library organized according to the sections described in XX.

References

Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Image Analysis: 13th Scandinavian Conference, SCIA 2003 Halmstad, Sweden, June 29–July 2, 2003 Proceedings 13*, pages 363–370. Springer, 2003.

Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023.

Michael J. D. Powell. An efficient method for finding the minimum of a function of several variables without calculating derivatives. *The Computer Journal*, 7 (2):155–162, 1964.